فصل ۶: struct، union، حافظهٔ پویا و لیست پیوندی

چرا این فصل؟ این فصل پل بین برنامه‌سازی و ساختمان داده است. تست‌ها معمولاً یکی از این‌ها را می‌پرسند: کپی شدن struct، اندازهٔ (padding) struct، `malloc` و `calloc`، و ردگیری لیست پیوندی. **فرض‌ها:** `int` ۴ بایتی، `double` و اشاره‌گر ۸ بایتی، تراز (alignment) پیش‌فرض gcc. **اعتبار:** خروجی همهٔ کدها با `gcc` اجرا و چک شده است. **زمان مطالعه:** یک بلوک ۹۰ دقیقه‌ای.

درس ۱: struct

```
struct P { int x, y; };
struct P a = {1, 2}, b;
b = a;            // کپی می‌شود کل struct
b.x = 9;          // a.x همان ۱ می‌ماند
```

- دسترسی: `a.x` برای متغیر struct و `p->x` (یعنی `(*p).x`) برای اشاره‌گر به struct.
- **انتساب struct کپی کامل است**، حتی اگر داخلش آرایه باشد. آرایهٔ معمولی را نمی‌شود با `=` کپی کرد، ولی آرایهٔ داخل struct همراه struct کپی می‌شود.
- **ارسال struct به تابع با مقدار است** (کپی). برای تغییرش اشاره‌گر بفرست: `void mv2(struct P *p) { p->x = 100; }`.
- آرایه‌ای از struct: `p++` به struct بعدی می‌رود و `(p+1)->x` فیلد عضو بعدی است.

درس ۲: اندازهٔ struct و union

قاعدهٔ padding

۱. هر فیلد باید در آدرسی مضرب **اندازهٔ خودش** شروع شود (char: ۱، int: ۴، double: ۸).

۲. اندازهٔ کل struct به مضرب **بزرگ‌ترین تراز** فیلدهایش گرد می‌شود.

struct	چینش	sizeof
`{char c; int i; char d;}`	`c` + ۳ پر کن + `i` (۴) + `d` + ۳ پر کن	۱۲
`{int i; char c; char d;}`	`i` (۴) + `c` + `d` + ۲ پر کن	۸
`{char c; double d;}`	`c` + ۷ پر کن + `d` (۸)	۱۶

نتیجه: ترتیب فیلدها اندازه را عوض می‌کند. فیلدهای بزرگ را اول بگذار.

union: همهٔ فیلدها در **یک** حافظه شروع می‌شوند. اندازه = بزرگ‌ترین فیلد، گردشده به تراز. مثلاً `union {int i; char c[8];}` ۸ بایت است.

درس ۳: حافظهٔ پویا

تابع	کار
`malloc(n)`	n بایت می‌دهد؛ **مقدار اولیه ندارد** (زباله)
`calloc(k, size)`	k عضو می‌دهد، **همه صفر**
`realloc(p, n)`	اندازه را عوض می‌کند و مقادیر قبلی حفظ می‌شوند (ممکن است آدرس عوض شود)
`free(p)`	آزاد می‌کند

سه خطای کلاسیک

- **نشت حافظه:** آدرس حافظه را گم کنی بدون اینکه free کنی (مثلاً p را قبل از free به جای دیگری اشاره بدهی).
- **اشاره‌گر آویزان:** بعد از `free(p)`، استفاده از p یا هر کپی‌اش (`q = p`) رفتار تعریف‌نشده است.
- **free دوباره:** رفتار تعریف‌نشده.

درس ۴: لیست پیوندی در C

```
struct node { int v; struct node *next; };

struct node *push(struct node *h, int v) {          // درج در ابتدا
    struct node *n = malloc(sizeof *n);
    n->v = v;  n->next = h;  return n;
}
int count(struct node *h) { return h ? 1 + count(h->next) : 0; }

struct node *rev(struct node *h) {                  // وارون در O(n)
    struct node *prev = NULL;
    while (h) { struct node *nx = h->next; h->next = prev; prev = h; h = nx; }
    return prev;
}
```

درج ۱، ۲ و ۳ در ابتدا، لیست `3 → 2 → 1` را می‌سازد (ترتیب برعکس ورود). بعد از `rev` می‌شود `1 → 2 → 3`.

تمرین فصل ۶

۱. `sizeof(struct {char a; char b; int c;})` چند است؟

۲. با `struct P arr[3] = {{1,2},{3,4},{5,6}}; struct P *p = arr; p++;`، مقدار `p->y` و `(p+1)->x` چیست؟

> 🔑 **جواب‌ها:** پاسخ‌نامهٔ انتهای فصل، بخش ۱ (درس ۴: لیست پیوندی در C)

📝 آزمون فصل ۶

> تست‌های **تألیفی**. خروجی همه با `gcc` تأیید شده است.

زمان: ۱۲ دقیقه.

۱.

```
struct P { int x, y; };
struct P a = {1, 2}, b;  b = a;  b.x = 9;
printf("%d %d", a.x, b.x);
```

۱) `9 9` ۲) `1 9` ۳) `1 1` ۴) خطای کامپایل

۲.

```
void mv(struct P p)   { p.x = 100; }
void mv2(struct P *p) { p->x = 100; }
// main:
struct P a = {1, 2};  mv(a);  printf("%d ", a.x);  mv2(&a);  printf("%d", a.x);
```

۱) `100 100` ۲) `1 1` ۳) `1 100` ۴) `100 1`

۳. `sizeof(struct S { char c; int i; char d; })` ؟ ۱) ۶ ۲) ۸ ۳) ۱۲ ۴) ۱۶

۴. `sizeof(struct T { int i; char c; char d; })` ؟ ۱) ۶ ۲) ۸ ۳) ۱۲ ۴) ۱۶

۵. `sizeof(union U { int i; char c[8]; })` ؟ ۱) ۴ ۲) ۸ ۳) ۱۲ ۴) ۱۶

۶.

```
int *p = calloc(4, sizeof(int));  p[1] = 5;
printf("%d", p[0] + p[1] + p[3]);
```

۱) ۵ ۲) مقدار نامعلوم ۳) ۰ ۴) ۱۵

۷. بعد از `h = push(push(push(NULL, 1), 2), 3)` (تابع push درس ۴)، پیمایش لیست از h چه چاپ می‌کند؟ ۱) `3 2 1` ۲) `1 2 3` ۳) `3 2 1` ۴) `3 1`

۸.

```
struct A { int v[3]; };
struct A a = {{1, 2, 3}}, b = a;  b.v[0] = 9;
printf("%d %d", a.v[0], b.v[0]);
```

۱) `9 9` ۲) `9 1` ۳) خطای کامپایل (آرایه کپی نمی‌شود) ۴) `1 1`

۹.

```
struct P arr[3] = {{1,2},{3,4},{5,6}};  struct P *p = arr;  p++;
printf("%d %d", p->y, (p + 1)->x);
```

۱) `2 3` ۲) `4 5` ۳) `3 4` ۴) `4 6`

۱۰.

```
int *p = malloc(3 * sizeof(int));
for (int i = 0; i < 3; i++) p[i] = i * i;
p = realloc(p, 5 * sizeof(int));  p[4] = 7;
printf("%d %d", p[2], p[4]);
```

۱) `4 7` ۲) `0 7` ۳) مقدار نامعلوم برای `p[2]` ۴) `2 7`

> 🔑 **جواب‌ها و راه‌حل:** پاسخ‌نامهٔ انتهای فصل، بخش ۲ (آزمون فصل ۶)

🗂 کارت خلاصهٔ فصل ۶

- انتساب و ارسال struct کپی است (حتی آرایهٔ داخلش). برای تغییر، اشاره‌گر بفرست و `->` بزن.
- padding: هر فیلد روی مضرب اندازهٔ خودش؛ کل struct مضرب بزرگ‌ترین تراز. `{char,int,char} = 12` و `{int,char,char} = 8`.
- union = بزرگ‌ترین فیلد. malloc زباله می‌دهد و calloc صفر. realloc مقادیر را حفظ می‌کند. بعد از free استفاده نکن.
- درج در ابتدای لیست ترتیب را برعکس می‌کند. وارون کردن با سه اشاره‌گر `prev` ، `h` و `nx` در `O(n)` .

🔑 پاسخ‌نامهٔ فصل

بخش ۱: درس ۴: لیست پیوندی در C

۱. `8` (`a`، `b`، ۲ بایت پر کن، `c`)

۲. `4 5`

بخش ۲: آزمون فصل ۶

سؤال	۱	۲	۳	۴	۵	۶	۷	۸	۹	۱۰
جواب	۲	۳	۳	۲	۲	۱	۲	۲	۲	۱

۱. انتساب struct کپی است و a دست نمی‌خورد.

۲. mv روی کپی کار می‌کند و mv2 از طریق اشاره‌گر، خود a را عوض می‌کند.

۳. `4 + 1 + 3 (پر کن) = 12 + (پر کن) 3 + 1`

۴. `8 = (پر کن) 2 + 1 + 1 + 4`

۵. بزرگ‌ترین فیلد ۸ بایت است.

۶. calloc صفر می‌کند: `5 = 0 + 5 + 0`.

۷. درج در ابتدا ترتیب را برعکس می‌کند: `1 2 3`.

۸. آرایهٔ داخل struct همراه struct کپی می‌شود.

۹. p به `arr[1] = {3,4}` اشاره می‌کند، پس `p->y = 4` و `(p+1)->x = 5`.

۱۰. realloc مقادیر قبلی را حفظ می‌کند: `p[2] = 4` و `p[4] = 7`.